



SOICT



Movies Linked Open Data

A 5-star knowledge graph of 250 films, linked to Wikidata and DBpedia

IT6390E Knowledge Graphs · Capstone project

Group 2: Đỗ Tuấn Minh · Nguyễn Tiến Đức · Hoàng Huy Chiến

School of Information and Communication Technology, Hanoi University of Science and Technology

10 October 2026

Agenda

01

Goal and the 5-star scheme

What the assignment asks for and how we measure it

Đức

02

Ontology and reasoning

Competency questions, classes, defined classes, Hermit, SWRL

Minh

03

Data and linking

TMDB data, RDF pipeline, links to Wikidata and DBpedia, link quality

Đức

04

Querying and live demo

Local and federated SPARQL, Protégé walkthrough, limitations

Minh

Five steps from a raw CSV to 5-star Linked Data



Step 1

Ontology

Core classes, an n-ary CastRole, reused vocabularies, defined classes the reasoner can fill



Step 2

Data

250 top-rated films from the Kaggle TMDB 5000 dataset



Step 3

RDF

Turtle generated with Python RDFLib, a URI for every film and person



4-star



Step 4

Links

920 owl:sameAs links to Wikidata, 912 to DBpedia



5-star



Step 5

SPARQL

Terminal interface and Protégé, 21 queries: local, cast-role and federated

The 5-star Linked Open Data scheme, applied



On the web under a licence

Repository on GitHub; RDF under CC BY-NC 4.0, code under MIT



Machine-readable structured data

The cleaned dataset is a CSV table, not a scan or a PDF



Non-proprietary format

CSV and Turtle, readable without any vendor software



URIs and W3C standards

A URI for every film, person, genre; RDF, OWL and SPARQL



Linked to other data

owl:sameAs: 920 links to Wikidata, 912 to DBpedia

Questions the knowledge graph must answer



Who directed movie X?

Answered by Query 1



Which movies feature actor Y?

Answered by Query 5



Which movies were released after year Y?

Answered by Query 2



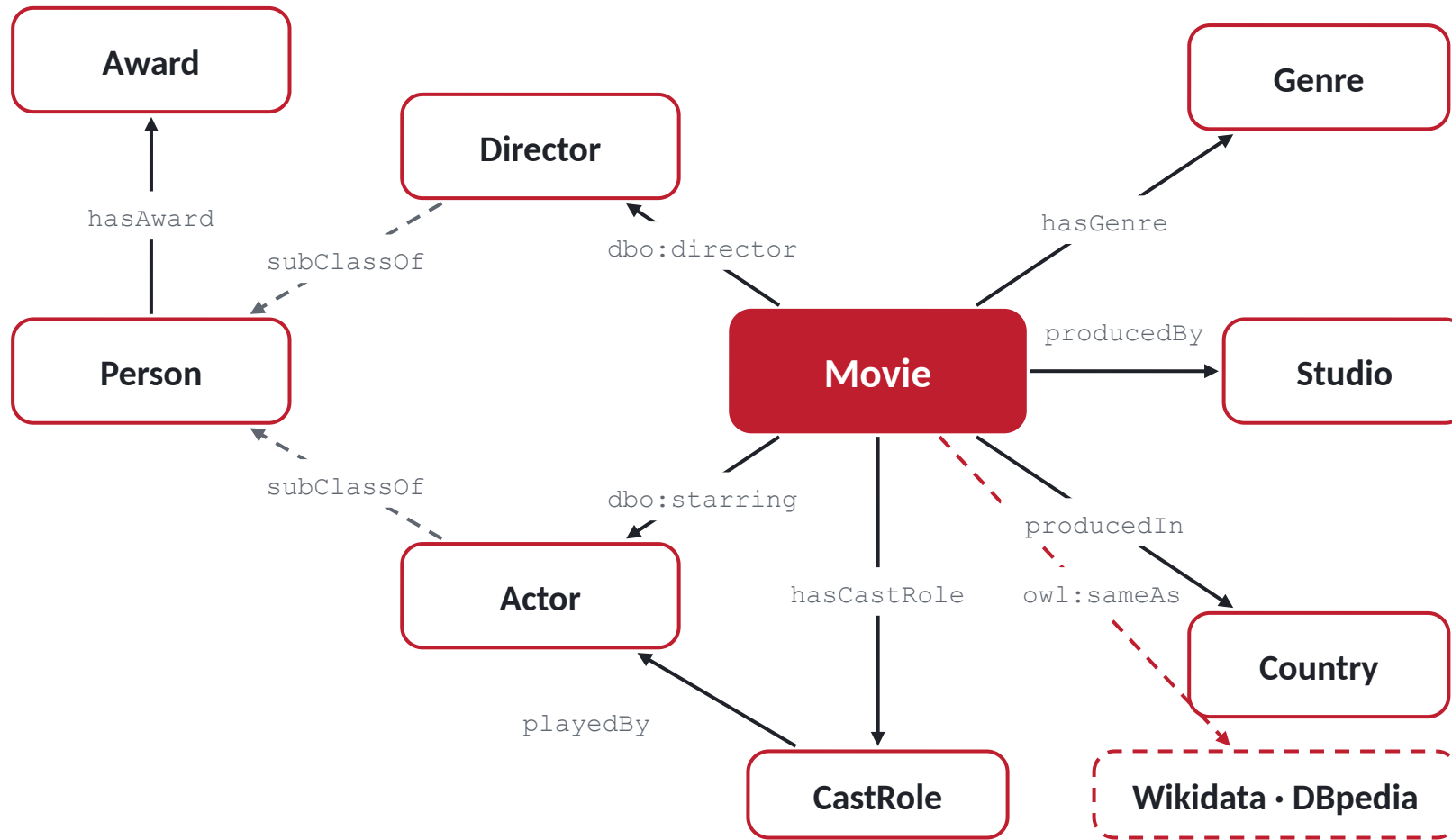
Which director has the most movies?

Answered by Query 3

Written first

Following Grüninger & Fox, the competency questions were fixed before modelling. Each one fixed which classes and properties the ontology needs, and each one became a SPARQL query in the demo.

Ontology: core classes, mostly reused vocabulary



Reused before minting

- `schema:Movie`, `foaf:Person`
- `dbo:director`, `dbo:starring`
- `dc:title`, `foaf:name`

Local terms

- `hasGenre`, `producedBy`, `producedIn`
- `releaseYear`, `hasAward`
- `Genre`, `Studio`, `Country`, `Award`
- `CastRole`: `characterName`, `billingOrder`

The reasoner fills classes nobody tagged by hand

29

ActionMovie

equivalentClass: hasGenre
value Action

16

ComedyMovie

Same pattern, disjointWith
ActionMovie

32

AwardWinningDirector

Director and hasAward some
Award (Oscar Best Director,
from Wikidata)

5

ActorDirector

SWRL rule: directed and
starred in the same movie



Counts from HermiT, run by scripts/check_reasoning.py. No movie or person in the data is typed with these four classes; every member above is inferred.

How the defined classes are written

hasValue restriction

Same idea as VegetarianPizza in pizza.owl:
membership follows from a property value.

```
ActionMovie ≡ hasGenre value genre_action  
ComedyMovie ≡ hasGenre value genre_comedy  
ComedyMovie disjointWith ActionMovie
```

Intersection + existential

Awards come from Wikidata: 32 Oscar
winners for Best Director in our data.

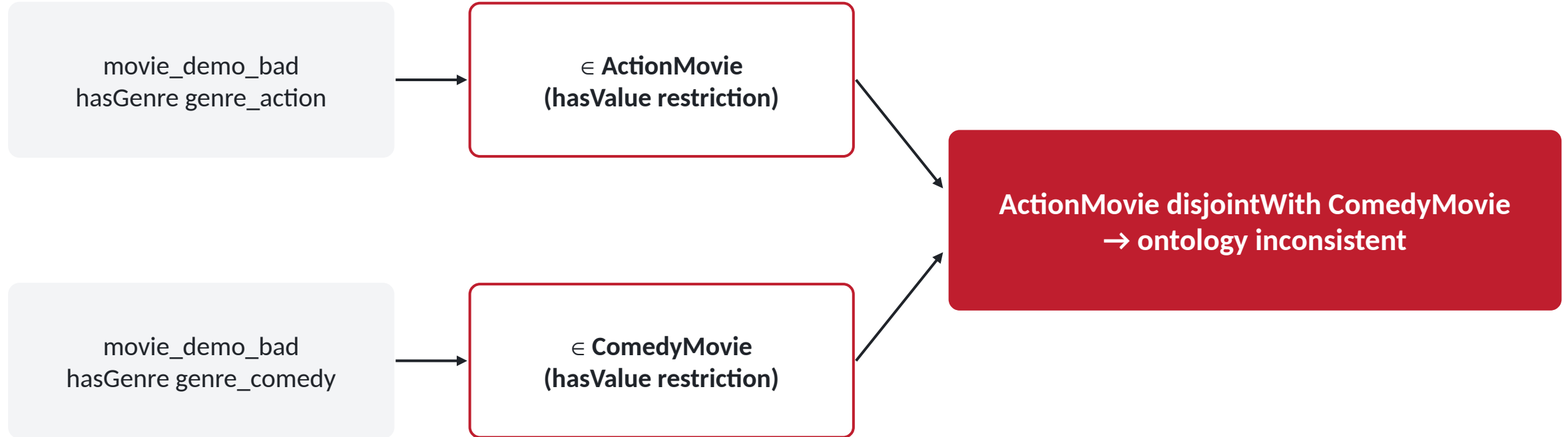
```
AwardWinningDirector ≡  
  Director and (hasAward some Award)
```

SWRL rule

Needed because OWL cannot require the
same person on two different properties of
one movie.

```
Movie(?m) ∧ dbo:director(?m, ?p)  
  ∧ dbo:starring(?m, ?p) → ActorDirector(?p)
```

Catching a contradiction with the reasoner



ontology/demo_inconsistency.ttl adds this one movie only during the demo. Protégé's Explain button shows exactly this justification, the same pattern as Russell's paradox in the lecture.

Data: 250 top-rated films from TMDb

250

movies

672

people: 156 directors, 522 actors

17

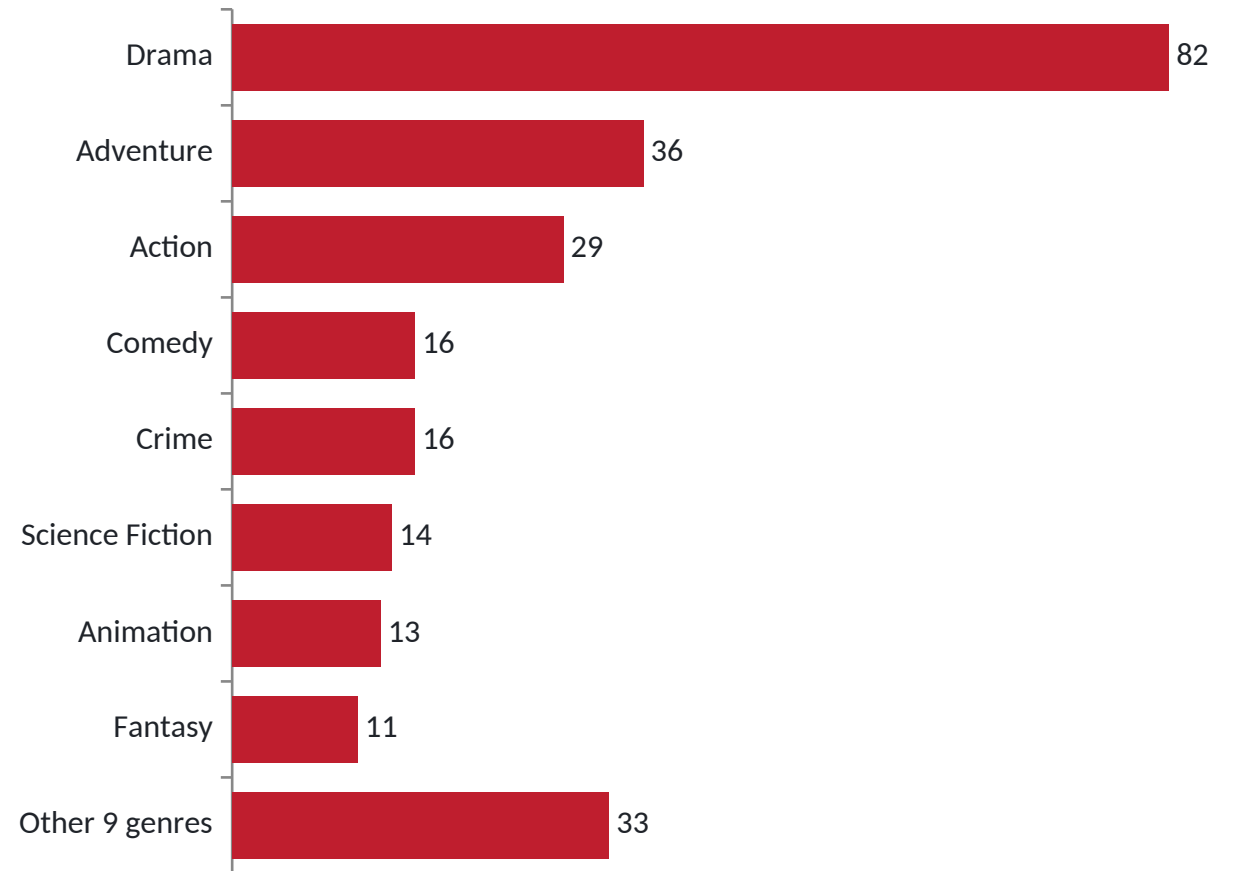
genres

99 / 16

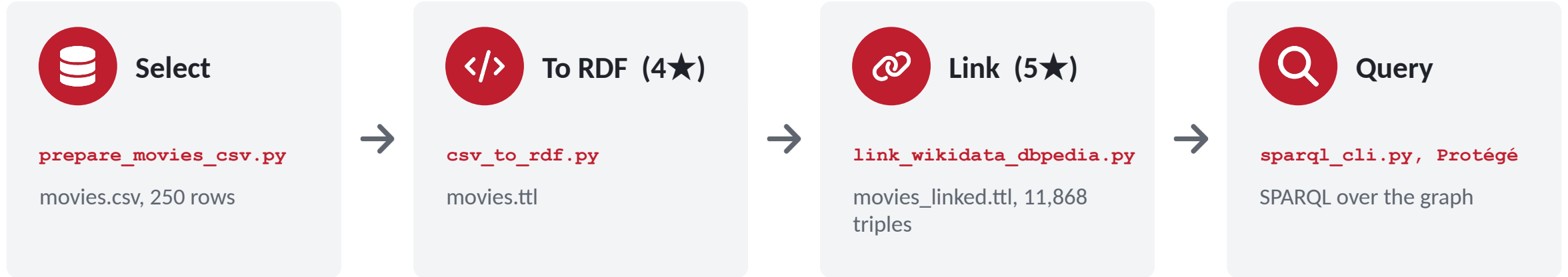
studios / countries

Source: Kaggle "TMDB 5000 Movie Dataset" (4,803 films). Kept films with at least 1,000 votes, then the 250 best rated. Director, top-3 cast, primary genre, studio and country per film. Uses the TMDb API but is not endorsed by TMDb.

Movies per primary genre



Pipeline: four scripts, every step reproducible



```
ex:movie/1124 a schema:Movie ;
  dc:title "The Prestige" ;   ont:releaseYear 2006 ;
  dbo:director ex:person/525 ;   ont:hasGenre ont:genre_drama ;
  ont:producedBy ont:studio/warner-bros ;
  owl:sameAs wd:Q46551 , dbr:The_Prestige_(film) .
```

Mapping CSV columns to RDF

CSV column	RDF	Vocabulary
id	<code>ex:movie/{id} a schema:Movie</code>	schema.org
title	<code>dc:title</code>	Dublin Core
year	<code>ont:releaseYear (xsd:integer)</code>	local
director_id, director_name	<code>dbo:director ex:person/{id}; foaf:name</code>	DBpedia, FOAF
cast_ids, cast_names, cast_characters	<code>dbo:starring; ont:CastRole (characterName, billingOrder)</code>	DBpedia, local
genre	<code>ont:hasGenre ont:genre_{name}</code>	local
studio	<code>ont:producedBy ont:studio/{slug}</code>	local
country	<code>ont:producedIn ont:country/{slug}</code>	local



URIs reuse the TMDB ids, so they stay stable across runs and double as the key for linking.

Linking: match on identifiers, not on titles



Title + year on DBpedia

First attempt matched 0 of 8 sample films: DBpedia film resources carry no `dbo:releaseDate`, so the year filter never matches. Titles are also ambiguous across remakes.



TMDB id on Wikidata

- Wikidata stores the TMDB id: P4947 for films, P4985 for people
- DBpedia via its own `owl:sameAs`, only `dbo:Film` or `dbo:Person`
- Ambiguous ids and redirect resources are left unlinked

920

links to Wikidata

912

links to DBpedia

249 / 250

movies linked to both

Checking the links, and a lesson on owl:sameAs

100%

Wikidata precision

30 of 30 links correct

96.7%

DBpedia precision

29 of 30 links correct

Random sample of 15 films and 15 people (seed 42); every Wikidata and DBpedia page opened and compared on year, director, occupation and birth date.



The wrong link was a redirect

DBpedia keeps the type and owl:sameAs of an article later merged into another one. Checking all 916 DBpedia links we had then found four that pointed at a different entity:

- Alakina Mann → the film The Others (2001)
- Joel Coen → Coen brothers
- Anthony Russo → Russo brothers
- Lilly Wachowski → The Wachowskis

Fix: redirects are skipped, and followed only for a plain rename. No redirect is left among the 912 links.

Describing the dataset itself



DCAT

Catalog, dataset with titles in English and Vietnamese, one distribution per Turtle file



VoID

Triples, class and property partitions, two linksets: 920 links to Wikidata, 912 to DBpedia



PROV-O

Derived from the Kaggle CSVs, generated by one pipeline activity using Wikidata, DBpedia and our scripts

Licences

Code (scripts/)	MIT
Our RDF and metadata	CC BY-NC 4.0
Upstream movie facts	TMDb terms of use
Wikidata links	CC0
DBpedia links	CC BY-SA

Generated by scripts/generate_metadata.py from the real files; it stops if the link counts disagree with link_report.csv.

SPARQL: local queries and federated ones

```
# Query 3: director with the most movies
SELECT ?directorName (COUNT(?movie) AS ?n)
WHERE {
    ?movie dbo:director ?d .
    ?d foaf:name ?directorName .
}
GROUP BY ?directorName
ORDER BY DESC(?n) LIMIT 5
```

Director	Movies
Steven Spielberg	8
Quentin Tarantino	7
Christopher Nolan	7
Martin Scorsese	6
David Fincher	6

5 local queries

one per competency question,
plus the linked-movies check

6 cast-role queries

characters and billing order
through the n-ary CastRole

10 federated queries

SERVICE calls to Wikidata and
DBpedia

Two interfaces

sparql_cli.py in a terminal, and
Protégé's SPARQL tab

Federated query: Wikidata fills the gaps

```
# Query 7 (abridged)
SELECT ?name ?movies ?born ?age WHERE {
  { # local: directors with 6+ movies
    ?d foaf:name ?name ;
      owl:sameAs ?wd . ... }
  SERVICE <https://query.wikidata.org/sparql> {
    ?wd wdt:P569 ?born .      # birth date
  }
  BIND(?firstYear - YEAR(?born) AS ?age)
}
```

Director	Movies	Born	Age at first
Steven Spielberg	8	1946	29 (1975)
Christopher Nolan	7	1970	30 (2000)
Quentin Tarantino	7	1963	29 (1992)
David Fincher	6	1962	33 (1995)
Martin Scorsese	6	1942	34 (1976)

"Age at first" = age at their earliest film in our 250-film dataset.



The 5th star at work: the query only reaches Wikidata because every director carries an owl:sameAs link to it.

Live demo

1 Load

Open movies.ttl in Protégé, merge the linked data and awards

2 Reason

Start Hermit: ActionMovie, ComedyMovie, AwardWinningDirector fill up

3 Query

Run the sample queries in the SPARQL tab

4 Federate

Run a SERVICE query to Wikidata from the terminal

5 Rule

Add the SWRL rule: 5 people become ActorDirector

6 Break it

Merge the bad movie: Hermit explains the inconsistency

Limitations and next steps

Roles modelled as subclasses

Director and Actor are roles, not rigid types (OntoClean). Kept as subclasses of Person for simplicity; a hasRole property would be cleaner.

One genre per movie

Only the primary genre is kept, so ActionMovie and ComedyMovie can stay disjoint without making real action comedies inconsistent.

URIs not dereferenceable

Resources live under example.org. Publishing them, for example behind Fuseki, would make the dataset fully browsable Linked Data.

Snapshot data, plain-string characters

250 films; one film and one actor stay unlinked as ambiguous. characterName is a plain string, so one name can mean different characters (query 18).



SOICT



Thank you. Questions?

github.com/minhdo2207/it6390e-lod-capstone

Đỗ Tuấn Minh · ontology, reasoning, demo

Nguyễn Tiến Đức · data, RDF conversion, linking

Hoàng Huy Chiến · metadata, federated queries, report and video